

```
"""
STEP 2: Import necessary libraries

chromadb → used to create vector database
SentenceTransformer → used to generate embeddings
"""

import chromadb
from sentence_transformers import SentenceTransformer
```

 $\frac{1}{4}$

```
"""
```

```
STEP 4: Load embedding model
```

```
This model converts text into numerical vectors (embeddings)
that capture semantic meaning.
```

```
"""
```

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
```

```
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
modules.json: 100% 349/349 [00:00<00:00, 26.7kB/s]
```

```
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 11.4kB/s]
```

```
README.md: 10.5k/? [00:00<00:00, 776kB/s]
```

```
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.73kB/s]
```

```
config.json: 100% 612/612 [00:00<00:00, 55.0kB/s]
```

```
model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 452MB/s]
```

```
Loading weights: 100% 103/103 [00:00<00:00, 430.55it/s, Materializing param=pooler.dense.weight]
```

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
```

```
Key | Status | |
-----+-----+--
embeddings.position_ids | UNEXPECTED | |
```

```
Notes:
```

```
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
```

```
tokenizer_config.json: 100% 350/350 [00:00<00:00, 33.8kB/s]
```

```
vocab.txt: 232k/? [00:00<00:00, 14.3MB/s]
```

```
tokenizer.json: 466k/? [00:00<00:00, 23.0MB/s]
```

```
special_tokens_map.json: 100% 112/112 [00:00<00:00, 10.4kB/s]
```

```
config.json: 100% 190/190 [00:00<00:00, 16.7kB/s]
```

```
"""
```

```
STEP 5: Prepare sample text dataset
```

```
- documents → list of text data
- ids → unique identifiers for each document
```

```
"""
```

```
documents = [
    "Artificial Intelligence is the simulation of human intelligence.",
    "Machine Learning is a subset of AI.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing deals with text data.",
    "RAG combines retrieval and generation.",
    "AI is used in healthcare and automation.",
    "Neural networks are inspired by the human brain.",
    "Supervised learning uses labeled data.",
    "Unsupervised learning finds hidden patterns.",
    "Reinforcement learning learns through rewards."
]
```

```
# Generate IDs like doc1, doc2, ...
```

```
ids = [f"doc{i}" for i in range(1, len(documents)+1)]
```

```
"""
```

```
STEP 6: Convert text into embeddings
```

```
Each document is converted into a vector representation.
These vectors are used for similarity comparison.
```

```
"""
```

```
embeddings = model.encode(documents) # Convert all documents into vectors
```

```
"""
```

```
STEP 7: Store embeddings in ChromaDB
```

```

We store:
- documents (text)
- embeddings (vectors)
- ids (unique keys)
"""

collection.add(
    documents=documents,    # Original text data
    embeddings=embeddings, # Vector representation
    ids=ids                # Unique IDs
)

```

```

"""
STEP 8: Perform similarity search

- Convert query into embedding
- Search for most similar documents in database
"""

```

```
query = "What is Artificial Intelligence?"
```

```
# Convert query to embedding
query_embedding = model.encode([query])
```

```
# Retrieve top 3 similar documents
results = collection.query(
    query_embeddings=query_embedding,
    n_results=3
)

```

```
print(results) # Raw output
```

```
ds': [['doc1', 'doc6', 'doc2']], 'embeddings': None, 'documents': [['Artificial Intelligence is the simulation of human inte
```

```

"""
STEP 9: Display retrieved documents

Extract and print only the relevant documents
from the query results.
"""

```

```
print("Top Matching Documents:\n")
```

```
for doc in results['documents'][0]: # Access retrieved docs
    print("-", doc)
```

```
Top Matching Documents:
```

- Artificial Intelligence is the simulation of human intelligence.
- AI is used in healthcare and automation.
- Machine Learning is a subset of AI.

```

"""
STEP 10: Create a reusable retrieval function

```

```

This function:
- Takes a query as input
- Returns top similar documents
"""

```

```

def retrieve(query):
    """
    Retrieve top 2 similar documents for a given query.

    Args:
        query (str): User input query

    Returns:
        list: Top matching documents
    """

    query_embedding = model.encode([query]) # Convert query to vector

    results = collection.query(
        query_embeddings=query_embedding,
        n_results=2
    )

    return results['documents'][0]

```

```
# Test the function
print("\nRetrieved Results:\n")
print(retrieve("Explain machine learning"))
```

Retrieved Results:

```
['Machine Learning is a subset of AI.', 'Supervised learning uses labeled data.']
```

```
"""
BONUS: Display similarity scores (distance values)

Lower distance = higher similarity
"""

results = collection.query(
    query_embeddings=query_embedding,
    n_results=3
)

print("Documents with Scores:\n")

for doc, score in zip(results['documents'][0], results['distances'][0]):
    print(f"Score: {score:.4f} -> {doc}")
```

Documents with Scores:

```
Score: 0.3886 -> Artificial Intelligence is the simulation of human intelligence.
Score: 0.6098 -> AI is used in healthcare and automation.
Score: 0.7215 -> Machine Learning is a subset of AI.
```